

02_event_session

4. Event Model

The GA4 export is an **event stream**, not a table of business entities. Every row in `bigquery-public-data.ga4_obfuscated_sample_ecommerce.events_*` is **one event** — a single thing a user did (loaded a page, viewed a product, added to cart, paid). There is no session row, no order row, and no user row in the raw export; all of those are entities Helios *reconstructs* from the event stream. That is why the **event grain is the atomic foundation of the entire warehouse**: every fact and dimension downstream is a deterministic roll-up of these events, and if the event-access layer is wrong, every number above it is silently wrong. This section documents the event grain, its canonical taxonomy, the nested column shapes, the single flattening pattern Helios sanctions, and the two staging models (`stg_ga4_events` , `stg_ga4_event_params`) that are the only governed doorway between raw GA4 and everything else.

4.1 The event grain

property	value
Grain	one row per event (one user action at one microsecond timestamp)
Natural key	(<code>user_pseudo_id</code> , <code>event_timestamp</code> , <code>event_name</code> , <code>event_bundle_sequence_id</code>) — no enforced PK in raw
Partitioning	date-sharded tables <code>events_YYYYMMDD</code> (read via <code>_table_suffix</code>); spans <code>20201101</code> – <code>20210131</code>
Owner	Google / GA4 export (we own only the <i>access</i> layer, never the raw bytes)
Why it exists	the immutable source of truth ; every session, funnel flag, order, and item line is derived from it. We never query it directly except through staging.

A single visit ("session") is dozens of these event rows that happen to share a `ga_session_id` ; a single order is one (or, due to export retries, several) `purchase` rows that share a `transaction_id` . The event grain knows nothing about either — sessionization and order dedup are *our* logic, applied in the intermediate and mart layers. The event row is deliberately "dumb and complete": it carries scalar context (`device.*` , `geo.*` , `traffic_source.*`), nested `event_params` , a nested `items` array, and a scalar `ecommerce` struct, so that any entity we need can be reconstructed without going back to a different source.

4.2 Canonical event taxonomy

The table below catalogs the canonical `event_name` values in the dataset: when each fires, the load-bearing `event_params` it carries, and the macro-funnel stage it maps to. The seven **bolded** stages plus `session_start` constitute the session-scoped macro funnel (`session_start` → `view_item` → `add_to_cart` → `begin_checkout` → `add_shipping_info` → `add_payment_info` → `purchase`); the remaining events are engagement/instrumentation signals that inform `engaged_session` and landing-page derivation but are not funnel steps.

event_name	fires when	key event_params	funnel stage / role
<code>session_start</code>	first event of a session	<code>ga_session_id</code> , <code>ga_session_number</code>	session_start (funnel denominator anchor)
<code>first_visit</code>	first-ever event for a <code>user_pseudo_id</code> cookie	<code>ga_session_id</code>	drives <code>is_new_user</code> / <code>ga_session_number = 1</code>
<code>page_view</code>	any page load	<code>page_location</code> , <code>page_title</code> , <code>page_referrer</code>	engagement; source of <code>landing_page</code>
<code>view_promotion</code>	promo/banner impression	<code>items[]</code> , <code>promotion_id</code>	top-of-funnel (pre- <code>view_item</code>)
<code>view_item_list</code>	category / search-results / list page view	<code>items[]</code> , <code>item_list_name</code>	top-of-funnel (pre- <code>view_item</code>)
<code>view_item</code>	product detail page (PDP) view	<code>items[]</code> , <code>page_location</code>	view_item
<code>select_item</code>	item clicked within a list	<code>items[]</code> , <code>item_list_name</code>	micro: list → PDP
<code>add_to_cart</code>	item added to cart	<code>items[]</code> , <code>value</code> , <code>currency</code>	add_to_cart
<code>view_cart</code>	cart drawer / cart page viewed	<code>items[]</code> , <code>value</code>	micro: cart (between ATC and checkout)
<code>begin_checkout</code>	checkout initiated	<code>items[]</code> , <code>value</code> , <code>coupon</code>	begin_checkout
<code>add_shipping_info</code>	shipping tier entered	<code>shipping_tier</code> , <code>value</code>	add_shipping_info

event_name	fires when	key event_params	funnel stage / role
add_payment_info	payment method entered	payment_type , value	add_payment_info
purchase	order completed	transaction_id , value , items[] , full ecommerce struct	purchase
scroll	90% scroll depth reached	percent_scrolled	engagement
click	outbound / UI click	link_url , outbound	engagement
user_engagement	foreground-engagement heartbeat	engagement_time_msec , session_engaged	drives engaged_session / engagement_time_msec

Two honesty notes that ride along with this taxonomy: (1) the funnel built from these events is **session-scoped** and uses **max-downstream** semantics (Section 5), so `view_cart` , `select_item` , and `view_promotion` are deliberately *not* macro-funnel steps — they are micro-signals the Critic can test but are excluded from the canonical six `reached_*` flags. (2) `session_engaged` and `engagement_time_msec` arrive on `user_engagement` , `page_view` , and other events, so engagement is a **session-level aggregate** of these params, not a property of any single event.

4.3 The nested event-row structure

A raw GA4 event row mixes four shapes. Understanding them is what makes the "flatten once" rule (Section 4.5) necessary rather than optional.

```

EVENT ROW (one user action)
├─ scalar columns ..... user_pseudo_id, event_name, event_timestamp (micros),
                        event_date, event_bundle_sequence_id, user_id (≈ always NULL)
├─ event_params ..... ARRAY<STRUCT<key STRING,
                        value STRUCT<string_value, int_value, float_value, double_value>>>
                        → ga_session_id, ga_session_number, page_location, source, medium,
                        campaign, engagement_time_msec, session_engaged, value, ...
├─ items ..... ARRAY<STRUCT<item_id, item_name, item_brand, item_category,
                        item_category2..5, item_variant, price_in_usd, price,
                        quantity, item_revenue_in_usd, coupon>>
                        → present on view_item / add_to_cart / begin_checkout / purchase
├─ ecommerce ..... STRUCT< transaction_id, purchase_revenue_in_usd, purchase_revenue,
                        refund_value_in_usd, shipping_value_in_usd, tax_value_in_usd,
                        total_item_quantity, unique_items > (populated on purchase)
├─ device ..... STRUCT< category, operating_system, web_info.browser, ... >
├─ geo ..... STRUCT< country, region, city, ... >
└─ traffic_source ..... STRUCT< source, medium, name > ← USER FIRST-TOUCH (the gotcha, §5.3)

```

Why one row carries all four: GA4 ships a **denormalized, self-contained event**. The cost of that convenience is that the two most important keys in the whole system — `ga_session_id` (the session identity) and the session-scoped `source / medium` — are buried inside the `event_params` array and must be `UNNEST`-extracted before any join is possible. The `items` array and `ecommerce` struct are the revenue payload, populated only on `purchase` (and, for `items`, on the cart/checkout events). `device.* / geo.*` are scalar structs that need only field access, not unnesting. `traffic_source.*` is a scalar struct too — but a *dangerous* one, because it is user first-touch, not this-session source (Section 5.3).

4.4 The `get_event_param` `UNNEST` pattern

`event_params` is an array, so a key like `ga_session_id` is not a column — it is an element whose value lives in exactly one of four typed sub-fields (`string_value`, `int_value`, `float_value`, `double_value`). The governed extraction pattern is a **correlated scalar subquery** over `UNNEST(event_params)`, which returns one scalar per event row and avoids the row fan-out a `CROSS JOIN UNNEST` would cause:

```
-- scalar extractors: one value per event, no fan-out
(select ep.value.int_value   from unnest(event_params) ep where ep.key = 'ga_session_id') as
ga_session_id,
(select ep.value.string_value from unnest(event_params) ep where ep.key = 'page_location') as
page_location,
(select ep.value.double_value from unnest(event_params) ep where ep.key = 'engagement_time_msec') as
engagement_time_msec
```

This is standardized into the canonical `get_event_param` macro (`macros/get_event_param.sql`) so the typed-slot choice is made in exactly one place and never re-improvised:

```
{% macro get_event_param(key, type='string') %}
  (select ep.value.{{ type }}_value from unnest(event_params) ep where ep.key = '{{ key }}')
{% endmacro %}
-- usage: {{ get_event_param('ga_session_id','int') }} as ga_session_id
```

When a model needs to scan *many* param keys at once (rather than pluck a few known ones), `stg_ga4__event_params` provides the fully-exploded alternative — one row per (event, param key) — so callers never re- `UNNEST` the array themselves.

4.5 Staging models — `stg_ga4__events` and `stg_ga4__event_params`

Helios flattens the nested event row **exactly once**, in staging, and never again. Everything above staging reads typed scalar columns. This is both a correctness rule (one canonical typed-slot decision; no per-model re-`UNNEST` drift) and a cost rule (the expensive array work happens in one cheap, always-fresh `view` instead of being repeated in every downstream model).

stg_ga4__events

property	value
Layer	staging (materialized as <code>view</code>)
Grain	one row per event (1:1 with raw)
Primary key	surrogate <code>event_id</code> , or the natural key (<code>user_pseudo_id</code> , <code>event_timestamp</code> , <code>event_name</code>)
Foreign key	→ <code>src_ga4.events</code> (the raw source, via <code>source()</code>)
Owner / steward	analytics-eng / Platform
Why it exists	the 1:1 typed/renamed access layer : surfaces every scalar the warehouse needs — <code>session_key</code> , <code>ga_session_id</code> , <code>page_location</code> , <code>session source / medium</code> , <code>device.*</code> , <code>geo.*</code> , <code>ecommerce.transaction_id</code> , <code>purchase_revenue_in_usd</code> — so downstream models are isolated from raw GA4 schema drift and never touch the nested arrays.

```
-- stg_ga4__events : 1 row per event, scalars surfaced from the nested raw row
select
  to_hex(md5(user_pseudo_id || '-' || cast(
    {{ get_event_param('ga_session_id','int') }} as string))) as session_key,
  user_pseudo_id,
  {{ get_event_param('ga_session_id','int') }} as ga_session_id,
  {{ get_event_param('ga_session_number','int') }} as ga_session_number,
  event_name,
  event_timestamp,
  timestamp_micros(event_timestamp) as event_ts,
  parse_date('%Y%m%d', event_date) as event_date,
  {{ get_event_param('page_location') }} as page_location,
  {{ get_event_param('engagement_time_msec','int') }} as engagement_time_msec,
  {{ get_event_param('session_engaged') }} as session_engaged,
  -- session-scoped source/medium FIRST; user first-touch only as fallback (the gotcha, $5.3)
  coalesce({{ get_event_param('source') }}, traffic_source.source) as source,
  coalesce({{ get_event_param('medium') }}, traffic_source.medium) as medium,
  {{ get_event_param('campaign') }} as campaign,
  device.category as device_category,
  device.operating_system as operating_system,
  device.web_info.browser as browser,
  geo.country, geo.region,
  ecommerce.transaction_id as transaction_id,
  ecommerce.purchase_revenue_in_usd as purchase_revenue_in_usd
from {{ source('src_ga4','events') }}
where _table_suffix between '20201101' and '20210131'
```

`stg_ga4__events` does **only** renames, casts, and the single param-extraction — no business logic, no sessionization, no aggregation. That discipline is what lets it stay a cheap `view` and what makes raw-schema changes a one-file fix.

`stg_ga4__event_params`

property	value
Layer	staging (materialized as <code>view</code>)
Grain	one row per event × param key
Primary key	(<code>event natural key</code> , <code>param_key</code>)
Foreign key	→ <code>src_ga4.events</code>
Owner / steward	analytics-eng / Platform
Why it exists	flattens the nested <code>event_params</code> ARRAY once into a long table so no downstream model ever re-UNNESTs ; the multi-key / exploratory complement to the scalar extractors on <code>stg_ga4__events</code> .

```
-- stg_ga4__event_params : 1 row per (event, param key), all typed slots exposed
select
  to_hex(md5(user_pseudo_id || '-' || cast(event_timestamp as string) || '-' || event_name)) as
    event_key,
  user_pseudo_id, event_name, event_timestamp,
  ep.key          as param_key,
  ep.value.string_value as string_value,
  ep.value.int_value   as int_value,
  ep.value.float_value  as float_value,
  ep.value.double_value as double_value
from {{ source('src_ga4', 'events') }} , unnest(event_params) ep
```

Why flatten once, in staging

1. **Single typed-slot decision.** Choosing `int_value` vs `double_value` for a key is a correctness call; making it once (in the macro / staging) eliminates an entire class of silent type bugs.
2. **No fan-out drift.** The scalar-subquery pattern keeps `stg_ga4__events` 1:1 with raw; if every model re-unnested ad hoc, a stray `CROSS JOIN UNNEST` would multiply rows and quietly inflate counts.
3. **Cost.** The array scan is the expensive part of touching GA4. Doing it in one cheap `view` keeps the autonomous run inside its fixed byte budget rather than paying for the UNNEST in

every mart.

4. **Schema-drift isolation.** When GA4 renames a struct or moves a param, exactly one staging model changes; the marts and the semantic layer are untouched.

This is the structural reason the event model is the atomic foundation: **everything above staging composes typed scalars, never raw arrays.**

5. Session Model

GA4 ships events, not sessions. The **session** is the first entity Helios reconstructs, and it is the spine of the whole product: the macro funnel, every rate metric, engagement, and most dimensional slicing all live at session grain. This section defines sessionization precisely, then documents the five session-grain models — `int_ga4__sessionized`, `fct_sessions`, `fct_funnel`, `fct_daily_funnel`, and `fct_funnel_by_dim` — each with grain, primary key, foreign keys, owner/steward, a "why it exists" line, and full column intent.

5.1 What a session is, and the `session_key`

A **session** = `(user_pseudo_id, ga_session_id)`. `user_pseudo_id` is the device/cookie identifier; `ga_session_id` is GA4's per-cookie session counter, carried inside `event_params`. The canonical surrogate — used identically everywhere, never re-improvised — is:

```
session_key = TO_HEX(MD5(CONCAT(user_pseudo_id, '-', CAST(ga_session_id AS STRING))))
-- sessions = COUNT(DISTINCT session_key) -- never FARM_FINGERPRINT, never COUNT(*)
```

GA4 starts a new `ga_session_id` after 30 minutes of inactivity **and** at UTC midnight, so one human visit that straddles midnight splits into two sessions. A session is attributed to the **day of its first event** (`min(event_timestamp)`), so a midnight-crossing session counts once, on its start day. Rows with a NULL `ga_session_id` cannot be sessionized and are dropped at the intermediate layer.

Honesty note (cookie-grain identity). The user key is `user_pseudo_id`, a **device + browser cookie**, not a person. `user_id` is almost always NULL in this obfuscated export, so **no cross-device stitching** is possible: one human on phone + laptop is two users, and cookie churn (clearing cookies, Safari ITP / Firefox ETP ~7-day client-cookie caps, incognito) re-mints the same human as a new user. Every user-grain count inherits this bias; the Critic always caveats `users / new_users / returning_users / ARPU` as cookie approximations, never true person counts.

5.2 Derived session attributes

Sessionization reconstructs, per `(user_pseudo_id, ga_session_id)` group, the attributes GA4 never ships as a row:

- **landing_page** — the `page_location` of the **earliest-timestamp event** carrying one (typically `session_start` / `first_visit` / the first `page_view`).
- **session_start_micros** / **session_end_micros** — min / max of `event_timestamp`; `date_key = DATE(TIMESTAMP_MICROS(session_start_micros))`.
- **is_new_user** — `ga_session_number = 1` (the cookie's first session); equivalently the session that contains `first_visit`.
- **ga_session_number** — the session ordinal for the cookie, bucketed downstream into `session_number_bucket`.
- **device_category** / **operating_system** / **browser** / **country** / **region** — `any_value` within the session (stable per visit).
- **engaged_session** — see §5.4.
- **channel_group** — derived from session `source` / `medium` via the single `channel_group_case()` macro (`macros/channel_group.sql`), the only place channel logic lives, producing exactly the 10 GA4 default groups.

5.3 Session-scoped source/medium and the `traffic_source` first-touch FALLBACK gotcha

This is the single most error-prone attribution detail in the dataset. The scalar `traffic_source` struct on every event is **USER FIRST-TOUCH** — the source/medium that *originally acquired the cookie*, repeated unchanged on every later event regardless of how the current session actually arrived. Using it as "the session's channel" mis-attributes every returning session to its original acquisition channel.

The governed rule: **prefer the session-scoped `event_params.source` / `medium`** (which reflect how *this* session arrived), and **fall back to the user-level `traffic_source` struct only when the session params are NULL**:

```
coalesce({{ get_event_param('source') }}, traffic_source.source, '(direct)') as source,  
coalesce({{ get_event_param('medium') }}, traffic_source.medium, '(none)') as medium
```

This is the only sanctioned source/medium path. `channel_group` then flows from these session-scoped values through `channel_group_case()`. Carried caveat: because attribution is session-scoped occurrence, channel-sliced metrics answer "which channel was *this session* on", not "which channel deserves credit" — there is no last-non-direct or data-driven model, and (critically) no cost data, so ROI/ROAS is impossible from this layer.

5.4 Engaged session and new user

- **engaged_session** = `session_engaged = '1'` OR `engagement_time_msec ≥ 10000` — GA4's own engaged-session bar (10+ seconds, or a conversion / multiple pageviews), aggregated to session grain (`LOGICAL_OR` of the flag; `MAX` of engagement time). It is a non-bounce / attention proxy, **not** funnel progress: an engaged session need not view a product, and `engagement_rate = engaged_sessions / sessions`.
- **is_new_user** = `ga_session_number = 1`. Honesty note: "new" is cookie-first-session, not first-time-human; cookie churn systematically inflates `new_users`.

5.5 `int_ga4_sessionized` — the sessionization keystone

property	value
Layer	intermediate (ephemeral / view ; not exposed to BI)
Grain	one row per session
Primary key	<code>session_key</code>
Foreign keys	built from <code>stg_ga4__events</code> (no mart FKs yet)
Owner / steward	analytics-eng
Why it exists	reconstructs the session row GA4 never ships by grouping events on <code>(user_pseudo_id, ga_session_id)</code> ; derives <code>landing_page</code> , session-scoped <code>source / medium</code> (with the <code>traffic_source</code> first-touch fallback), <code>channel_group</code> , <code>device/geo</code> , <code>engaged_session</code> , <code>is_new_user</code> , <code>ga_session_number</code> . KEYSTONE — if sessionization is wrong, every downstream number is silently wrong.

```
-- int_ga4_sessionized : 1 row per (user_pseudo_id, ga_session_id)
select
  to_hex(md5(user_pseudo_id || '-' || cast(ga_session_id as string))) as
    session_key,
  user_pseudo_id, ga_session_id,
  any_value(ga_session_number) as
    ga_session_number,
  min(event_timestamp) as
    session_start_micros,
  max(event_timestamp) as
    session_end_micros,
  array_agg(page_location ignore nulls order by event_timestamp limit 1)[safe_offset(0)] as
    landing_page,
  -- session-scoped source/medium with user first-touch FALLBACK (the gotcha)
  coalesce(array_agg(source ignore nulls order by event_timestamp limit 1)[safe_offset(0)],
    '(direct)') as source,
  coalesce(array_agg(medium ignore nulls order by event_timestamp limit 1)[safe_offset(0)],
    '(none)') as medium,
  any_value(device_category) as device_category, any_value(operating_system) as operating_system,
```

```

any_value(browser) as browser, any_value(country) as country, any_value(region) as region,
countif(true) as
    event_count,
max(coalesce(engagement_time_msec, 0)) as
    engagement_time_msec,
logical_or(session_engaged = '1') as
    session_engaged_flag
from {{ ref('stg_ga4__events') }}
where ga_session_id is not null
group by user_pseudo_id, ga_session_id

```

5.6 The `reached_*` MAX-DOWNSTREAM monotonic flags

The funnel is computed in `int_ga4__funnel_steps` (session grain, PK `session_key`, FK → `int_ga4__sessionized`; the second **KEYSTONE**) and carried onto `fct_funnel`. Each of the six `reached_*` flags uses **max-downstream** semantics: a session **reached stage X** if it **fi**red **X or any later funnel-stage event** during the visit. Rolling each flag forward to every downstream stage makes the funnel **monotonic by construction**:

```

sessions ≥ reached_view_item ≥ reached_add_to_cart ≥ reached_begin_checkout
          ≥ reached_add_shipping_info ≥ reached_add_payment_info ≥ reached_purchase

```

Because every downstream flag implies all upstream flags, a later-stage count can **never exceed** an earlier one, which is exactly what guarantees every step rate is ≤ 1 (e.g. `view_to_cart_rate = add_to_cart_sessions / view_item_sessions ≤ 1`). It also avoids dropping legitimate journeys that, say, re-add a cart item without re-firing `view_item`. (The retired `did_*` names are forbidden; an ordered/strict variant exists for the Critic to test step-skipping hypotheses but is not the default.) The canonical expression:

```

-- int_ga4__funnel_steps : 1 row per session, max-downstream (monotonic) flags
select
  to_hex(md5(user_pseudo_id || '-' || cast(ga_session_id as string)))
  as session_key,
  user_pseudo_id,
  true
  as did_session_start,
  logical_or(event_name in
    ('view_item', 'add_to_cart', 'begin_checkout', 'add_shipping_info', 'add_payment_info', 'purchase'))
  as reached_view_item,
  logical_or(event_name in
    ('add_to_cart', 'begin_checkout', 'add_shipping_info', 'add_payment_info', 'purchase'))
  as reached_add_to_cart,
  logical_or(event_name in ('begin_checkout', 'add_shipping_info', 'add_payment_info', 'purchase'))
  as reached_begin_checkout,
  logical_or(event_name in ('add_shipping_info', 'add_payment_info', 'purchase'))
  as reached_add_shipping_info,
  logical_or(event_name in ('add_payment_info', 'purchase'))
  as reached_add_payment_info,
  logical_or(event_name = 'purchase')
  as reached_purchase
from {{ ref('stg_ga4__events') }}

```

```
where ga_session_id is not null
group by user_pseudo_id, ga_session_id
```

5.7 fct_sessions — the conformed session entity-of-record

property	value
Grain	SESSION — one row per session
Primary key	session_key
Foreign keys	user_pseudo_id → dim_users.user_key ; date_key → dim_date.date_key ; channel_key → dim_channels.channel_key
Owner / steward	analytics-eng / Product Analytics
Why it exists	the conformed SESSION entity-of-record : the wide session-dimension row (engagement + descriptive dims + funnel reach) that all session-grain analysis conforms to. Carries engaged_sessions .

column	type	key	intent
session_key	STRING	PK	`TO_HEX(MD5(user_pseudo_id
user_pseudo_id	STRING	FK → dim_users	device/cookie key (de-facto user)
ga_session_id	INT64		session id within the cookie
date_key	DATE	FK → dim_date	session start date (from min(event_timestamp))
session_start_ts / session_end_ts	TIMESTAMP		first / last event timestamp of the session
channel_key	STRING	FK → dim_channels	session-scoped channel-group surrogate
source / medium / campaign	STRING		session-scoped acquisition (first-touch fallback)
landing_page	STRING		first page_location of the session

column	type	key	intent
device_category / operating_system / browser	STRING		session device descriptors
country / region	STRING		session geo descriptors
is_new_user	BOOL		ga_session_number = 1
ga_session_number	INT64		session ordinal for the cookie
event_count	INT64		events in the session
engaged_session	BOOL		session_engaged = '1' OR engagement_time_msec ≥ 10000
engagement_time_msec	INT64		summed engagement time

5.8 fct_funnel — the semantic layer's primary session grain

property	value
Grain	SESSION — one row per session
Primary key	session_key (also FK → fct_sessions)
Foreign keys	→ dim_users (user_pseudo_id), dim_date (date_key), dim_channels (channel_key)
Owner / steward	analytics-eng / Product Analytics
Why it exists	the PRIMARY session grain the semantic layer queries : the monotonic reached_* flags + deduped session_revenue + wide session dims, so sessions, users, the funnel rates, conversion, and revenue_per_session all resolve here. fct_funnel EXTENDS fct_sessions 1:1.

column	type	key	intent
session_key	STRING	PK / FK → fct_sessions	session id (1:1 with fct_sessions)
user_pseudo_id	STRING	FK → dim_users	user key

column	type	key	intent
date_key	DATE	FK → dim_date	session date
channel_key	STRING	FK → dim_channels	channel group
device_category / country / is_new_user			wide dims, denormalized so the semantic layer slices without joins
did_session_start	BOOL		always TRUE — the denominator anchor (= sessions)
reached_view_item	BOOL		reached view_item or any later stage (max-downstream)
reached_add_to_cart	BOOL		reached add_to_cart or beyond
reached_begin_checkout	BOOL		reached begin_checkout or beyond
reached_add_shipping_info	BOOL		reached add_shipping_info or beyond
reached_add_payment_info	BOOL		reached add_payment_info or beyond
reached_purchase	BOOL		fired purchase
session_revenue	FLOAT64		SUM(purchase_revenue_in_usd) in the session, deduped to one value per transaction_id ; 0 for non-purchasing sessions

The 1:1 `fct_sessions` ↔ `fct_funnel` split is deliberate: `fct_sessions` is the descriptive session entity (engagement, dims), while `fct_funnel` carries the funnel + revenue payload at the same grain. The semantic layer points its `session`, `user`, and `revenue` (session-attributed) metrics at `fct_funnel`, computing every rate as $\text{SUM}(\text{numerator}) / \text{SUM}(\text{denominator})$ over additive counts (e.g. `session_conversion_rate = COUNTIF(reached_purchase) / COUNT(*)`) so rates re-aggregate correctly across any dimension the Decompose agent slices.

5.9 `fct_daily_funnel` — pre-aggregated additive daily funnel

property	value
Grain	DAY × [channel_group , device_category , country , is_new_user]

property	value
Primary key	daily_funnel_key (md5 of the grain columns)
Foreign keys	date_key → dim_date ; channel_key → dim_channels
Owner / steward	analytics-eng / Growth Analytics
Why it exists	additive pre-aggregated daily funnel counts + revenue ; the feed for Monitor (time-series anomaly), Decompose (mix-shift), and the eval injector. Aggregates fct_funnel (which carries session_revenue) — rates are NOT stored , they recompute in the semantic layer so they stay re-aggregatable across any slice.

column	type	intent
daily_funnel_key	STRING	md5 of grain columns (PK)
date_key	DATE	day
channel_key	STRING	channel group
device_category / country / is_new_user		grain dimensions
sessions	INT64	COUNT(DISTINCT session_key)
users / new_users / returning_users	INT64	distinct users (all / new / returning)
engaged_sessions	INT64	sessions where engaged_session
view_item_sessions	INT64	SUM(reached_view_item)
add_to_cart_sessions	INT64	SUM(reached_add_to_cart)
begin_checkout_sessions	INT64	SUM(reached_begin_checkout)
add_shipping_info_sessions	INT64	SUM(reached_add_shipping_info)
add_payment_info_sessions	INT64	SUM(reached_add_payment_info)
purchasing_sessions	INT64	SUM(reached_purchase)
transactions	INT64	distinct transaction_id
revenue	FLOAT64	SUM(session_revenue)

Storing only additive counts (never rates) is the Simpson's-paradox defense: any rate the agents need is `SUM(num)/SUM(den)` recomputed at the requested grain, so a daily-grain table rolls up to week/month or to any dimension subset without re-deriving rates from pre-divided ratios.

5.10 `fct_funnel_by_dim` — single-dimension funnel rollup

property	value
Grain	DAY × DIMENSION (one canonical dimension at a time)
Primary key	composite <code>(date_key, dimension, dimension_value)</code>
Foreign keys	<code>date_key</code> → <code>dim_date</code>
Owner / steward	analytics-eng / Growth Analytics
Why it exists	the funnel rolled up by a single canonical dimension (long/unpivoted: a <code>dimension</code> name + <code>dimension_value</code>); the direct input to the mix-vs-rate decomposition (<code>decompose_change</code>).

column	type	intent
<code>date_key</code>	DATE	day (PK part)
<code>dimension</code>	STRING	the dimension name (e.g. <code>channel_group</code> , <code>device_category</code> , <code>country</code>) — PK part
<code>dimension_value</code>	STRING	the value within that dimension — PK part
<code>sessions</code>	INT64	mix weight <code>w_i</code> for <code>decompose_change</code>
<code>view_item_sessions ...</code> <code>purchasing_sessions</code>	INT64	the same additive <code>reached_*</code> counts as <code>fct_daily_funnel</code>
<code>transactions</code>	INT64	distinct <code>transaction_id</code>
<code>revenue</code>	FLOAT64	<code>SUM(session_revenue)</code>

Both `fct_daily_funnel` and `fct_funnel_by_dim` are **aggregations of `fct_funnel`, not FK children of it** — they materialize the same session-grain counts at coarser, agent-friendly grains. `fct_funnel_by_dim` deliberately holds one dimension at a time so the Decompose agent gets a clean `(w_i, r_i)` table per dimension: `sessions` is the mix weight, the `reached_*` counts give per-segment rates, and `decompose_change` splits any movement into mix / rate / interaction.

5.11 The session model as the analytical spine

Every model in this section traces back to one reconstruction — grouping events by `(user_pseudo_id, ga_session_id)` — and serves a distinct purpose: `int_ga4__sessionized` rebuilds the missing session row; `int_ga4__funnel_steps` adds the monotonic reach; `fct_sessions` is the conformed descriptive entity-of-record; `fct_funnel` is the 1:1 funnel + revenue extension that the **semantic layer queries directly**; and `fct_daily_funnel` / `fct_funnel_by_dim` pre-aggregate the same additive counts for Monitor, Decompose, and the eval injector. Honesty carries through the whole chain: identity is **cookie-grain** (no cross-device stitching), channel is **session-scoped occurrence with a first-touch fallback** (not attributed credit, and no cost data for ROI), and the ~3-month export window (Nov 2020 – Jan 2021) bounds every "returning"/cohort claim built on top of these sessions.